Prompt de sistema con salida JSON estructurada (Bloque 10) / Construye tu banco de prompts y valida su consistencia [Actividad core]

✓ **Hecho:** Ver

Construye tu banco de prompts y valida su consistencia

Ya tienes las dos piezas de esta unidad funcionando por separado: prompts de sistema que separan instrucción de contenido, y una salida en **JSON** que tu código puede leer directamente, con manejo de errores para cuando algo sale distinto a lo esperado.

Esta actividad, la que cierra la **Unidad U1.5**, te pide juntar ambas piezas en algo reutilizable: un banco de prompts para tu miniaplicación, y evidencia real de que ese banco es consistente, no solo que funcionó una vez.

Prerrequisitos: No necesitas instalar nada nuevo: "requests" y "python-dotenv" ya están en tu entorno virtual desde la Unidad 1.2, y tu archivo ".env" con "GEMINI_API_KEY" ya está configurado desde el Bloque 4.

Lo que sí necesitas tener resuelto antes de empezar son las cuatro funciones que usa el script de esta actividad: "armar_peticion" y "enviar_peticion" (Bloque 4, extendidas en la lección 1 de esta unidad), y "extraer_texto" junto con "parsear_respuesta_json" (lección 2 de esta unidad). Ninguna se vuelve a definir aquí: se asume que ya las tienes funcionando en tu módulo.

🗂️ Qué es un banco de prompts, y por qué se valida

Un **banco de prompts** es, simplemente, **un conjunto organizado de combinaciones de prompt** de

sistema más ejemplo de entrada, pensadas para una tarea específica de tu miniaplicación

(clasificar, resumir, extraer datos, lo que corresponda a tu caso).

Tenerlo por escrito, en un solo lugar, evita que cada parte de tu código arme sus propias instrucciones de forma distinta y sin criterio común.

> ⚠️ Que un prompt haya funcionado una vez no significa que funcione siempre: el mismo modelo, con la misma entrada, puede responder con una estructura ligeramente distinta en una repetición y en otra.

Por eso esta actividad no termina en **"escribí los prompts"**: termina en "los probé varias veces cada uno, y esto es lo consistentes que resultaron".

🛠️ Qué vas a construir en esta actividad

El objetivo son tres piezas relacionadas entre sí, todas dentro de tu repositorio:



Un banco de al menos 3 prompts

Cada uno con un nombre, su prompt de sistema completo, un ejemplo de entrada de usuario, y la estructura **JSON** que se espera como salida (las claves y sus tipos).



Un script de prueba

Que ejecute cada prompt del banco varias veces (al menos 3 repeticiones por **prompt**) contra la **API** real, e intente interpretar cada respuesta como **JSON** con la estructura esperada.



Un registro de resultados

Que muestre, por cada prompt, cuántas de las repeticiones dieron una salida **JSON** válida y con la estructura correcta, y cuántas fallaron.

🧩 Un ejemplo de cómo estructurar el banco

No hace falta un formato específico, pero organizar el banco como una lista de diccionarios en **Python** (o un archivo **JSON** aparte, si prefieres separarlo del código) mantiene cada prompt completo y fácil de

un archivo **JSON** aparte, si prefieres separarlo del código) mantiene cada prompt completo y fácil de recorrer en el **script** de prueba:

```
BANCO_DE_PROMPTS = [
    {
        "nombre": "clasificar_sentimiento",
        "system_instruction": (
            "Responde únicamente con un JSON con las claves 'sentimiento' "
            "('positivo', 'negativo' o 'neutro') y 'confianza' (0 a 1)."
        ),
        "ejemplo_usuario": "El servicio llegó tarde pero el producto está perfecto.",
        "claves_esperadas": ["sentimiento", "confianza"],
        "response_schema": {
            "type": "OBJECT",
            "properties": {
                "sentimiento": { "type": "STRING" },
                "confianza": { "type": "NUMBER" }
            },
            "required": ["sentimiento", "confianza"]
        },
    },
    # ... los demás prompts de tu banco, con la misma estructura
]
```

La clave "response_schema" es opcional (podrías omitirla en un prompt donde todavía no la necesites): más abajo, el script de prueba se la pasa a "armar_peticion" tal cual la agregaste en la lección anterior, usando "prompt.get('response_schema')", que devuelve "None" si un prompt no la trae.

Los tres prompts de tu banco no tienen que resolver tareas distintas entre sí sin relación: pueden ser tres variaciones útiles para tu misma miniaplicación (por ejemplo, clasificar, extraer una lista de entidades, y generar un resumen corto), cada una con su propia estructura de salida esperada.

🧪 Cómo se prueba la consistencia

El script de prueba recorre el banco, y para cada **prompt** hace varias llamadas reales a la **API** con el mismo ejemplo de entrada, contando cuántas veces la respuesta pasó la validación (**JSON** válido y con las claves esperadas) usando la misma lógica de limpieza y validación que ya construiste en la lección anterior.

```
import json
import time

registro = {}

for prompt in BANCO_DE_PROMPTS:
    exitos = 0
    intentos = 3
    for intento in range(intentos):
        url, body = armar_peticion(
            prompt["ejemplo_usuario"],
            prompt["system_instruction"],
            prompt.get("response_schema"),
        )
        respuesta = enviar_peticion(url, body)
        texto = extraer_texto(respuesta)
        try:
            datos = parsear_respuesta_json(texto)
            if all(clave in datos for clave in prompt["claves_esperadas"]):
                exitos += 1
        except ValueError as error:
            print(f"  intento {intento + 1} falló: {error}")
        time.sleep(1)
    print(f"{prompt['nombre']}: {exitos}/{intentos} respuestas válidas")
    registro[prompt["nombre"] = {"exitos": exitos, "intentos": intentos}

with open("prompts/registro_resultados.json", "w", encoding="utf-8") as archivo:
    json.dump(registro, archivo, indent=2, ensure_ascii=False)
```

Dos cambios respecto a lo que viste en la lección anterior, ninguno nuevo conceptualmente: "url, body = armar_peticion(...)" desempaqueta la tupla antes de pasarla a "enviar_peticion", exactamente como ya lo venías haciendo; y el "except" ahora imprime el motivo del fallo en vez de ignorarlo en silencio, para que un intento fallido te diga algo útil en vez de solo restar del conteo. El "time.sleep(1)" espacia las llamadas: este script hace muchas peticiones reales seguidas (tres por prompt, por cada prompt de tu banco), y sin una pausa es más fácil toparte con el límite de cuota (error 429) que en las lecciones anteriores, donde hacías como mucho una petición de prueba a la vez.

Con tu banco de prompts real, la consola se vería aproximadamente así (los números exactos dependen de tus prompts y del modelo):

```
clasificar_sentimiento: 3/3 respuestas válidas
  intento 2 falló: La respuesta no tiene la estructura esperada.
```

```
extraer_entidades: 2/3 respuestas válidas
resumen_corto: 3/3 respuestas válidas
```

Al terminar, además de esto en pantalla, tu carpeta "prompts/" tiene un archivo nuevo, "registro_resultados.json", con los mismos números en formato estructurado: es el archivo que efectivamente vas a versionar como evidencia, no una captura de la consola.

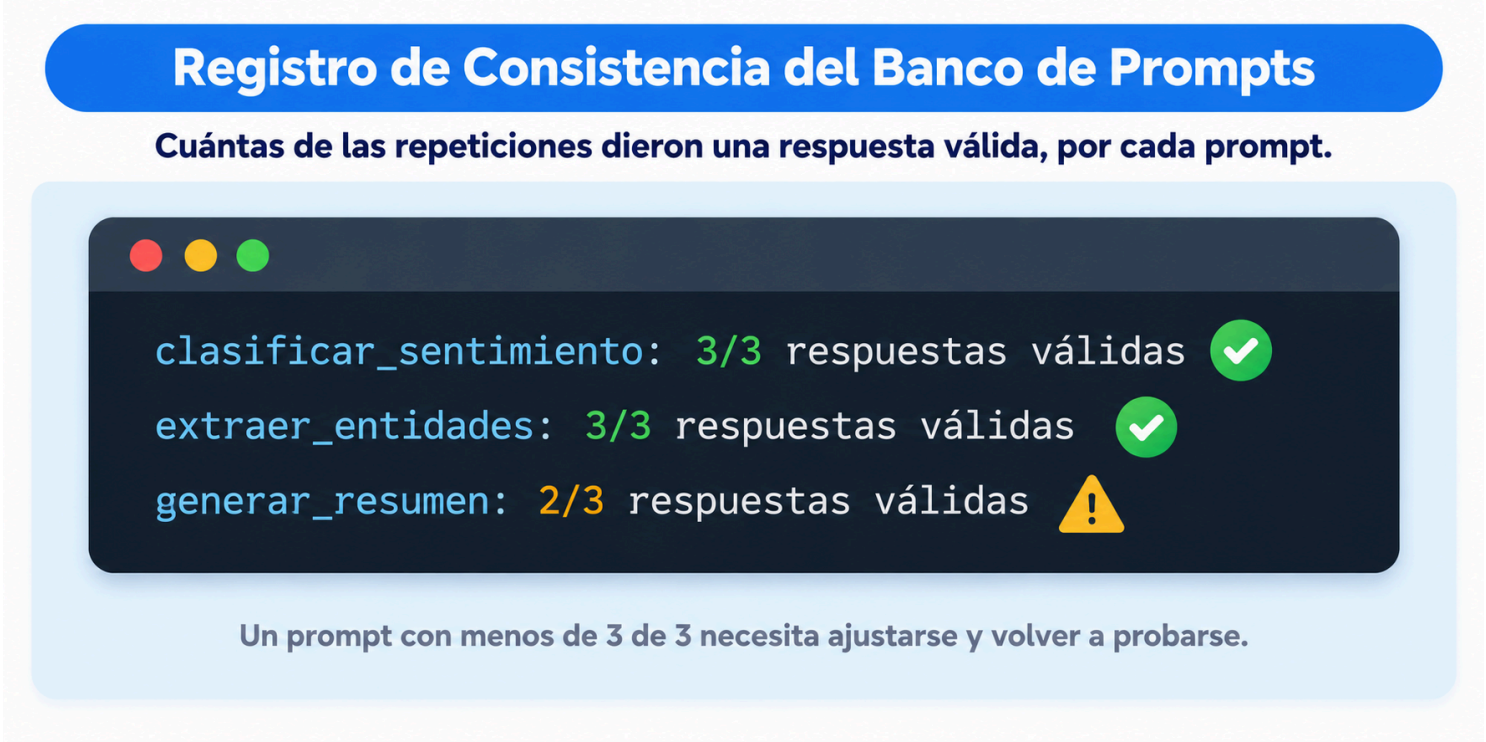
⚠️ **Posibles errores comunes**

Si ves varios errores **429** seguidos al correr este script, aunque ya tiene **"time.sleep(1)"** entre llamadas: es el mismo error de cuota que ya viste en la Unidad 1.4, solo que aquí es más probable por el volumen de peticiones (varios prompts, con varias repeticiones cada uno).

⏳ **Si te sigue pasando:** sube el tiempo de espera (por ejemplo a **2 o 3 segundos**) o reduce temporalmente cuántos prompts pruebas de una sola corrida.

Los errores de **JSON malformado** o de credenciales **(401)** no son nuevos de esta lección: siguen exactamente las mismas causas y soluciones que ya viste en la lección anterior de esta unidad y en la Unidad 1.3, respectivamente.

Tres repeticiones por prompt es el mínimo para esta actividad, no un techo: si te alcanza el tiempo y el presupuesto de tu cuenta, más repeticiones dan una medida de consistencia más confiable.



⚠️ Si un prompt falla más de lo esperado

Si alguno de tus prompts no llega a una consistencia razonable (por ejemplo, menos de 2 de 3 intentos válidos), no lo dejes así: ajusta la redacción de su **prompt** de sistema (instrucciones más específicas, un ejemplo del formato esperado dentro del propio **prompt**, o el uso de **"responseSchema"** si todavía no lo estabas usando para ese caso) y vuelve a correr la prueba.

➡️ Documenta ese ajuste: qué cambiaste, y cómo mejoró el resultado.

Cómo se entrega

Guarda dentro de tu repositorio, en una carpeta "prompts/" al mismo nivel que tu código, tres cosas: el banco de prompts (por ejemplo, en un archivo "prompts/banco.py", con la lista BANCO_DE_PROMPTS de más arriba), el script de prueba ("prompts/test_consistencia.py"), y el registro de resultados. Este último no necesitas inventarlo aparte: es el archivo "prompts/registro_resultados.json" que el propio script genera automáticamente cada vez que lo corres. Haz un commit específico para esta entrega, incluyendo los tres archivos.

```
git add prompts/
git commit -m "agrega banco de prompts y validación de consistencia"
git push
```

⚠️ Errores frecuentes en esta actividad



Probar cada prompt una sola vez

Una sola ejecución exitosa no demuestra consistencia; el objetivo explícito de esta actividad es medirla con repeticiones, no solo



Validar solo que sea JSON, no la estructura

Un **JSON** válido pero sin las claves esperadas debería contar como un intento fallido en tu registro, no como un éxito.



Dejar un prompt inconsistente sin ajustar

Si tu registro muestra que un prompt falla con frecuencia, la actividad espera que lo mejores y vuelvas a medir, no que

confirmar que el prompt **"funciona"**.

reportes el número bajo sin haber intentado corregirlo.

Siguiente lección

Bloque 11 · Preparación para Gate 1

Con tu banco de prompts probado y documentado, cierras la **Unidad U1.5**: de mezclar instrucciones dentro del mensaje del usuario, a separarlas en un prompt de sistema, forzar una salida estructurada, y ahora demostrar con datos que esa combinación es confiable de forma repetida.

Esa base de **prompt engineering** es exactamente lo que vas a necesitar para el **Gate 1**, que evalúa tu miniaplicación completa en la siguiente unidad.

Skillnest Learning Analytics

Bloque 10 - Construye tu banco de prompts y valida su consistencia [Core]

Tarea Core

Entregando como: Sebastián Eduardo Ayala Vásquez
ayala.phys.ds@gmail.com

Tu entrega

Subir archivo | Enlace (Drive, etc.)

Arrastra tu archivo aquí o haz clic para seleccionar

PDF, Word, Excel, PowerPoint, imágenes, ZIP — máx. 50 MB

Enviar entrega

Última modificación: miércoles, 7 de octubre de 2026, 17:02